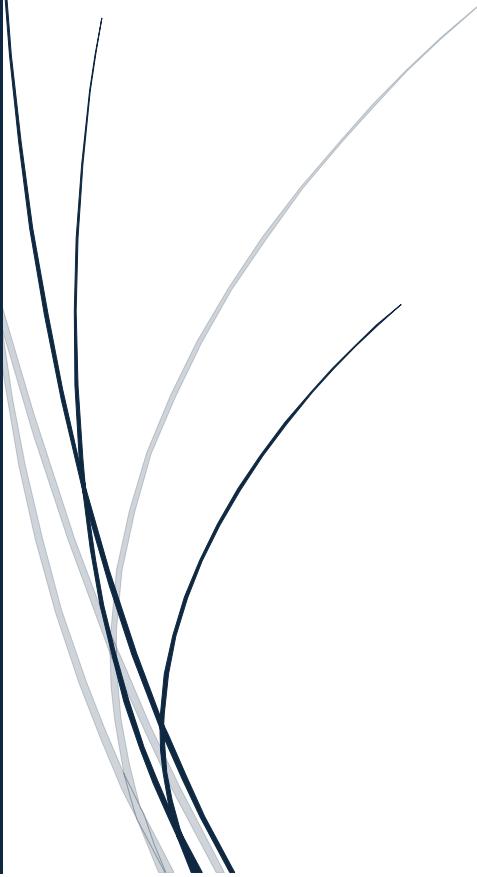




12/7/2025

Deploying a WAF with NGINX, ModSecurity, and CRS



Pedram Faramarz

Contents

1. Introduction	2
2. Architecture Diagram	2
3. Step-by-Step Implementation	2
Step 1 – Prepare folder structure and the files.....	2
Step 2 – Start the environment.....	6
Step 3 – Verify running containers.....	6
Step 4 – Check WAF is responding.....	7
4. Test Cases	7
5. Validation: Proving Blocked Requests	12
ModSecurity Web Application Firewall Log Analysis	12
Conclusion:	13

1. Introduction

In this lab, I deployed:

- A simple backend application (hashicorp/http-echo)
- An OWASP ModSecurity Core Rule Set (CRS) Web Application Firewall
- An NGINX reverse proxy with ModSecurity enabled
- Docker Compose for orchestration

I verified that:

- A safe request passes to the backend
- Malicious requests (SQLi, XSS, RCE patterns) are blocked by ModSecurity

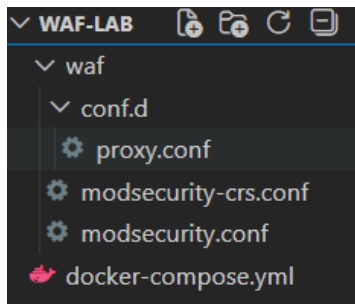
2. Architecture Diagram

[Browser] → [WAF (NGINX + ModSecurity + CRS)] → [App (hashicorp/http-echo)]

- WAF listens on port 8080
- Backend app listens on port 5678

3. Step-by-Step Implementation

Step 1 – Prepare folder structure and the files.



1.docker-compose.yml

This file orchestrates the deployment of the WAF and backend application using Docker Compose. Its main components:

- Services:
 - waf: Runs owasp/modsecurity-crs:nginx with ModSecurity enabled.
 - app: Runs hashicorp/http-echo as a simple HTTP backend.
- Ports:
 - Maps WAF container port 8080 to the host.
 - Maps backend app port 5678 internally.
- Volumes:
 - Mounts configuration files (ModSecurity and proxy settings) into the WAF container.

Purpose: Launch the full stack with a single command while keeping configurations modular and separate.

```

🔥 docker-compose.yml
  ▶ Run All Services
1  services:
  ▶ Run Service
2    app:
3      image: hashicorp/http-echo:0.2.3
4      command: ["-text=hello from app"]
5      ports:
6        - "5678:5678"
7      networks:
8        - waf-net
9
  ▶ Run Service
10   waf:
11     image: owasp/modsecurity-crs:nginx
12     ports:
13       - "8080:80"
14     volumes:
15       - ./waf/conf.d/proxy.conf:/etc/nginx/conf.d/proxy.conf:ro
16       - ./waf/modsecurity.conf:/etc/modsecurity/modsecurity.conf:ro
17       - ./waf/modsecurity-crs.conf:/etc/modsecurity/crs-setup.conf:ro
18
19     depends_on:
20       - app
21     networks:
22       - waf-net
23
24   networks:
25     waf-net:
26

```

2.modsecurity.conf

This is the main ModSecurity configuration file that defines how the WAF behaves.

Key features:

- Enables ModSecurity engine (SecRuleEngine On).
- Configures logging and detection modes (SecAuditEngine, SecAuditLog).
- Loads default ModSecurity directives like request body access, anomaly scoring, and security rules parsing.
- Sets rules for handling alerts and blocked requests.

Purpose: Core configuration controlling ModSecurity's operation within NGINX.

```
# waf/modsecurity.conf (Windows-safe, quick-start)

SecRuleEngine On
SecRequestBodyAccess On
SecResponseBodyAccess Off

# Disable file writes at startup by sending logs to /dev/null
SecAuditEngine RelevantOnly
SecAuditLog /dev/null
SecAuditLogParts ABIJDEFHZ
SecAuditLogType Serial

# Send debug to /dev/null to avoid file creation problems on Windows
SecDebugLog /dev/null
SecDebugLogLevel 0
```

3.modsecurity-crs.conf

This file integrates the OWASP Core Rule Set (CRS) with ModSecurity.

Key points:

- Includes CRS base rules for SQL injection, XSS, RCE, and other common attacks.
- Loads rule exclusions or customizations specific to your environment.
- Works alongside modsecurity.conf to enhance security without modifying the core WAF engine.

Purpose: Provides the actual security rules that detect and block malicious requests, making your WAF effective against attacks.

```
waf > ⚙ modsecurity-crs.conf
1  SecAction "id:900000,phase:1,pass,nolog,ctl:ruleEngine=On"
2  SecAction "id:900001,phase:1,pass,nolog,setvar:tx.crs_paranoia_level=1"
3
```

4.proxy.conf

Located in waf/conf.d/, this NGINX configuration handles reverse proxying to the backend app.

Key points:

- Forwards incoming requests from WAF port 8080 to the backend service (hashicorp/http-echo:5678).
- Integrates with ModSecurity to ensure traffic is inspected before reaching the backend.
- Sets basic NGINX parameters like proxy headers, buffering, and connection settings.

Purpose: Makes the backend service accessible through the WAF while allowing ModSecurity to filter malicious requests.

```
waf > conf.d > ⚙ proxy.conf
1  server {
2      listen 80;
3      server_name _;
4
5      modsecurity on;
6      modsecurity_rules_file /etc/modsecurity/modsecurity.conf;
7
8      location / {
9          proxy_pass http://app:5678/;
10         proxy_set_header Host $host;
11         proxy_set_header X-Real-IP $remote_addr;
12         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
13         proxy_http_version 1.1;
14         proxy_set_header Connection "";
15         proxy_buffering off;
16     }
17 }
18
```

Step 2 – Start the environment

```
PS C:\Users\pedra\Desktop\waf-lab> docker compose up -d --build
[+] Running 3/3
✓ Network waf-lab_waf-net Created
✓ Container waf-lab-app-1 Started
✓ Container waf-lab-waf-1 Started
```

Step 3 – Verify running containers

```
PS C:\Users\pedra\Desktop\waf-lab> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
f6f5dd66ea07	owasp/modsecurity-crs:nginx	"/docker-entrypoint..."	24 seconds ago	Up 23 seconds (health: starting)	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
9e952d11b48f	hashicorp/http-echo:0.2.3	"/http-echo '-text=h..."	24 seconds ago	Up 23 seconds	0.0.0.0:5678->5678/tcp, [::]:5678->5678/tcp

containers:

- owasp/modsecurity-crs:nginx
- hashicorp/http-echo

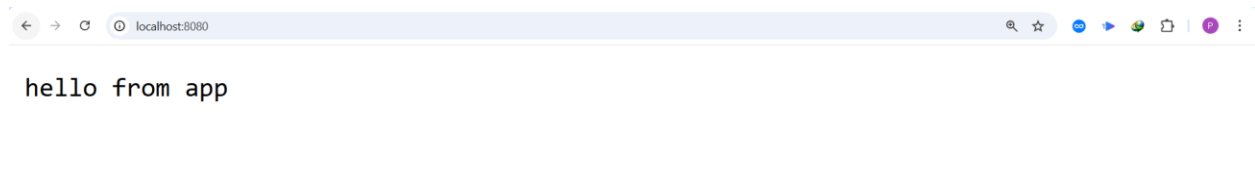
Step 4 – Check WAF is responding

Visit:

`http://localhost:8080`

Expected output:

hello from app



This proves the WAF is forwarding legitimate traffic.

4. Test Cases

Test Case 1 – Safe Request

Request:

`http://localhost:8080/?status=ok`

Expected:

✅ HTTP 200

hello from app

But this works in the app port.

The screenshot shows a web browser at the URL `localhost:5678/?id=1%27%20OR%20%271%27=%271`. The page content is "hello from app". The network inspector shows a successful GET request to `http://localhost:5678/?id=1%27%20OR%20%271%27=%271` with a status code of 200 OK. The response headers include `Content-Length: 15`, `Content-Type: text/plain; charset=utf-8`, `Date: Sat, 06 Dec 2025 23:05:59 GMT`, `X-App-Name: http-echo`, and `X-App-Version: 0.2.3`.

Test Case 3 – XSS Attempt

Request:

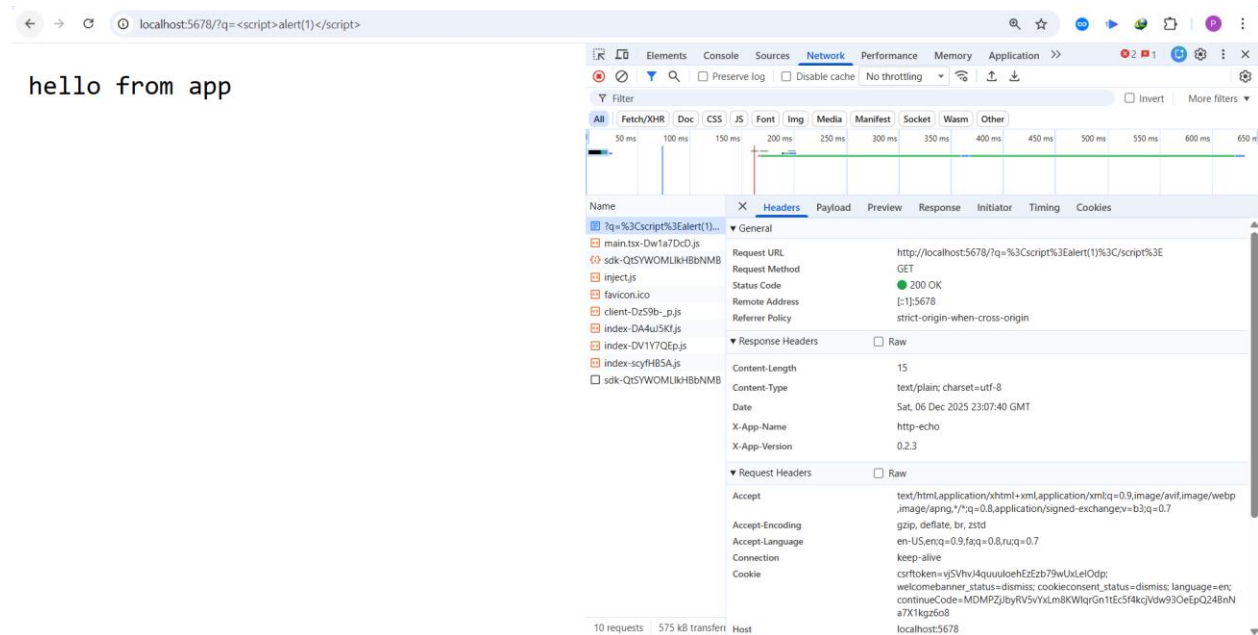
`http://localhost:8080/?q=<script>alert(1)</script>`

Expected:

✗ 403 Forbidden

The screenshot shows a web browser at the URL `localhost:8080/?q=%3Cscript%3Ealert(1)%3C%3E`. The page content is "403 Forbidden". The network inspector shows a 403 Forbidden response from `http://localhost:8080/?q=%3Cscript%3Ealert(1)%3C%3E` with a status code of 403 Forbidden. The response headers include `Connection: keep-alive`, `Content-Length: 548`, `Content-Type: text/html`, `Date: Sat, 06 Dec 2025 23:08:00 GMT`, and `Server: nginx`.

But it works on App port.



Test Case 4 – Remote Command Injection Attempt

Request:

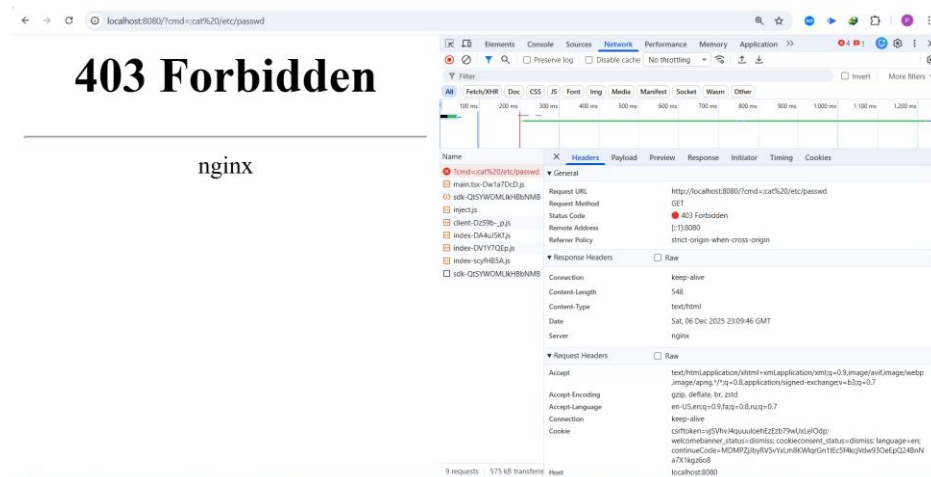
`http://localhost:8080/?cmd=;cat /etc/passwd`

Or:

`http://localhost:8080/?cmd=$(id)`

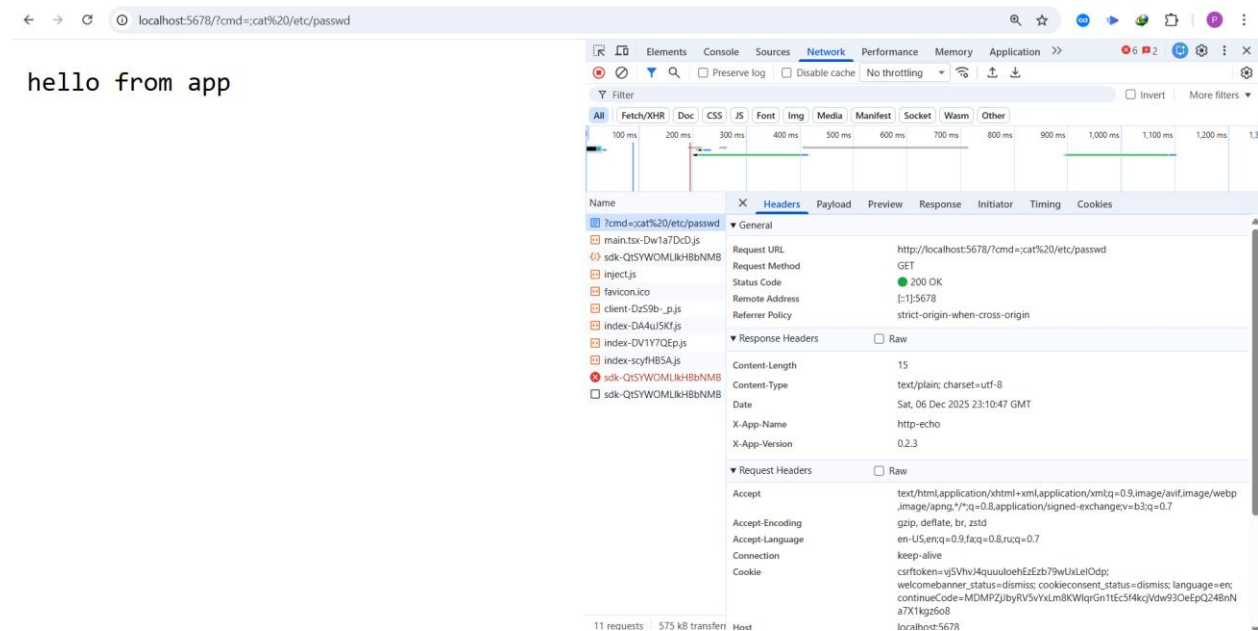
Expected:

✗ 403 Forbidden



But it works on App port.

hello from app



The screenshot shows a web browser at `localhost:5678/?cmd=:cat%20/etc/passwd`. The page displays "hello from app". The network inspector shows a successful GET request to `http://localhost:5678/?cmd=:cat%20/etc/passwd` with a status code of 200 OK. The response headers include `Content-Type: text/plain; charset=utf-8` and `X-App-Version: 0.2.3`.

Test Case 5 – Body-Based Attack (POST)

Command:

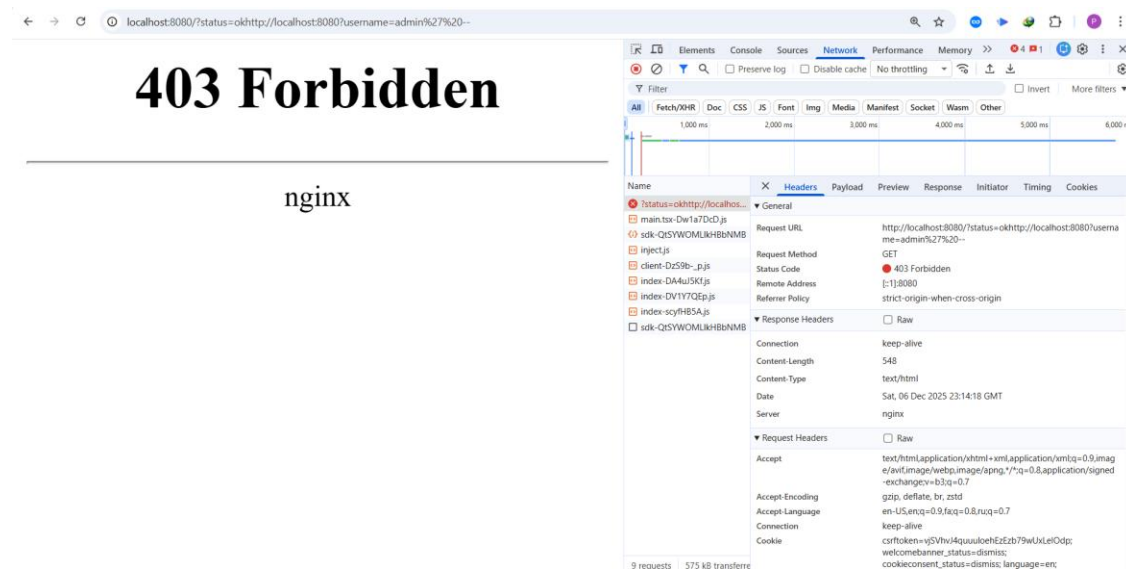
<http://localhost:8080?username=admin%27%20-->

Expected:

✗ 403 Forbidden (SQLi)

403 Forbidden

nginx



The screenshot shows a web browser at `localhost:8080/?status=okhttp://localhost:8080?username=admin%27%20--`. The page displays "403 Forbidden" and "nginx". The network inspector shows a 403 Forbidden response to a GET request to `http://localhost:8080/?status=okhttp://localhost:8080?username=admin%27%20--`. The response headers include `Content-Type: text/html` and `Server: nginx`.

5. Validation: Proving Blocked Requests

[Containers](#) / waf-lab-waf-1[illegible]

ModSecurity Web Application Firewall Log Analysis

Time (UTC)	Method	URI	Status	Anomaly Score	Attack Detected	Rules Triggered
23:06:51	GET	/?q=1' OR '1'=1'	❌ DENIED (403)	10	SQL Injection	942100, 942190, 942200, 942260, 942270
23:06:51	GET	/.well-known/appspecific/com.chrome.devtools.json	✅ ACCEPTED (200)	0	None	None
23:06:51	GET	/?q=<script>alert(1)</script>	❌ DENIED (403)	15	XSS Attack	941100, 941110, 941160
23:08:00	GET	/.well-known/appspecific/com.chrome.devtools.json	✅ ACCEPTED (200)	0	None	None
23:09:46	GET	/?cmd=;cat /etc/passwd	❌ DENIED (403)	15	RCE/LFI Attack	930120, 932100, 932160
23:09:46	GET	/.well-known/appspecific/com.chrome.devtools.json	✅ ACCEPTED (200)	0	None	None
23:14:18	GET	/?status=okhttp://localhost:8080?username=admin' --	❌ DENIED (403)	5	SQL Injection	942100

23:14:18	GET	/.well-known/appspecific/com.chrome.devtools.json	✅ ACCEPTED (200)	0	None	None
----------	-----	---	------------------	---	------	------

Conclusion:

In this lab I deployed a simple backend application together with a Web Application Firewall using NGINX, ModSecurity, and the OWASP Core Rule Set. The backend worked normally and the WAF was placed in front of it to filter all incoming traffic. I tested different types of requests to see how the WAF reacts. A normal safe request passed through the WAF and reached the application without any problems. When I sent SQL injection, XSS, and command-execution attempts, the WAF blocked all of them and returned a 403 error. I also checked the ModSecurity logs, and they clearly showed which rules were triggered and why the requests were blocked.

All tests showed that the WAF works correctly. Safe traffic is allowed and malicious traffic is stopped before it can reach the backend. This proves that the deployment and configuration of the WAF were successful and provide effective protection for the application.